ASSIGNMENT 1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Student Name: Madaraboina Gayathri

Register Number: 192472074

Course: C++ / Data Structures and Algorithms

Assignment: DSA01 – Assignment 1

Compiler: g++ with C++17 standard (enter exact g++ version before submission)

GitHub Repository: DSA0117-192472074-Assignment1
(https://github.com/GAYATHRI190506/DSA0117-C-.git)

Department / Institution: SIMATS ENGINEERING

Submission Date: 06-11-2026

Declaration

I declare that this assignment report documents the analysis, implementation and validation work carried out for the Metro Smart-Card fare system assignment. The report is organized according to Tasks 1–12 and includes the design decisions, pseudocode, test cases, complexity analysis and evidence expected by the assignment.

Assignment Overview

The assignment evaluates the use of C++ object-oriented programming principles, data types, constants, operators, implicit conversions, control structures, input/output handling, testing and design evaluation through a City Metro Smart-Card fare system.

Item	Specification
Network zones	Inner, Middle, Outer
Base fares	Rs 10, Rs 20, Rs 30
Peak periods	8–10 AM and 5–8 PM
Peak surcharge	20%
Minimum post-journey balance	Rs 50
Stored value	Integer paise
Passenger concession	Student / senior concession
Design challenge	Airport Rs 60, 10% weekend discount, monthly pass
Expected complexity	O(1) per journey and O(1) memory per card

Repository and Submission Structure

Folder	Main files / evidence
Task01	task01.cpp, analysis evidence
Task02	task02.cpp, input.txt, output.txt, Task02_Output.png
Task03	task03.cpp, input.txt, output.txt, Task03_Output.png
Task04	task04.cpp, input.txt, output.txt, Task04_Output.png
Task05	task05.cpp, input.txt, output.txt, Task05_Output.png
Task06	task06.cpp, input.txt, output.txt, Task06_Output.png
Task07	procedural.cpp, oop.cpp, input.txt, output.txt, screenshots
Task08	task08.cpp, input.txt, output.txt, Task08_Output.png
Task09	task09.cpp, test_cases.txt, output.txt, screenshots
Task10	task10.cpp, output.txt, Task10_Output.png
Task11	task11.cpp, output.txt, Task11_Output.png
Task12	task12.cpp, input.txt, output.txt, Task12_Output.png

Task 1 – Understand the Problem and Identify the Objects

The system is naturally modeled as an object-oriented fare-processing system because the problem contains entities with state and behaviour. A SmartCard owns a balance, a Passenger has a category, a Journey contains travel information, a Zone identifies the destination area, and a FareTable provides the monetary rules.

Object	Important data	Behaviour
SmartCard	card identifier, balance	check balance, check eligibility, deduct fare
Passenger	passenger type	identify regular/student/senior category
Journey	zone, time, journey status	calculate fare and validate travel
Zone	Inner/Middle/Outer	identify applicable base fare
FareTable	base fares, surcharge, minimum balance	provide fare constants/rules

OOP Principles

- Encapsulation: SmartCard balance should remain private and be changed through controlled operations.
- Abstraction: Fare calculation exposes the required operation without exposing all implementation details.
- Inheritance: A Passenger base type could be extended by Student and Senior passenger types if required.
- Polymorphism: Different passenger types can provide different concession behaviour through a common interface.

Correct-Journey Rules

- Zone must be valid.
- Fare must be calculated from a single authoritative fare definition.
- Peak surcharge must be applied only in the defined peak intervals.
- The fare must be deducted only when the balance after deduction is at least Rs 50.
- The balance must never become negative.
- Concession rules must be applied consistently.

Procedural Alternative

A procedural implementation can use global or local variables and free functions such as getFare(), isPeakHour(), canTravel() and deductFare(). The time per journey remains O(1) and the memory per card remains O(1), but rule changes can require edits in multiple functions or duplicated data locations. The OOP design localizes responsibilities and improves maintainability.

Pseudocode:

```
START
  Read zone, passenger type, balance and time
  Validate all inputs
  Determine base fare from zone
  Determine whether the time is peak
  Apply peak surcharge or concession according to the rule order
  Check whether balance - fare >= minimum balance
  IF valid
      deduct fare
      print journey as allowed
  ELSE
      reject journey
END
```

Complexity: O(1) time per journey; O(1) additional memory per card.

Task 2 – Tokens, Keywords, Identifiers and Constants

The fare table should be defined in one place so that a fare revision does not require searching through multiple program locations.

Token / concept	Example	Purpose
Keyword	switch, case, return, class	Reserved language constructs
Identifier	baseFare, balance, selectedZone	Program-defined names
Constant	3000, 20, 5000	Fixed values
Operator	+, -, >=, &&	Performs computation or comparison
Punctuator	{ }, ;, (,)	Structures the C++ program

- constexpr is preferred for compile-time fare constants because it is type-safe and scoped.
- const creates a read-only object but is not always required to be a compile-time constant.
- #define is textual substitution and provides weaker type and scope checking.
- enum class is safer than a plain enum because its values are scoped and do not implicitly convert to int in the same way.
- Identifiers should use meaningful names such as MIN_BALANCE_PAISE and PEAK_SURCHARGE_PERCENT.

Alternative: #define macros can avoid storage, but they lose type checking and scope semantics. Runtime performance is not meaningfully improved by using macros.

Complexity: O(1) lookup for a fixed fare selection; constants generally require no per-journey storage.

Evidence files:

```
Metro Fare Information
Base fare: Rs 30.0
Peak surcharge: 20%
Minimum balance required: Rs 50
```

Task 3 – Choose Data Types and Check Type Compatibility

Value	Recommended type	Reason
Fare / balance in paise	long long	Exact integer arithmetic and large range
Peak flag	bool	Two-state condition
Passenger category	char or enum class	Small categorical value
Hour	int	Range 0–23
Zone	enum class	Type-safe category
Journey count	long long	Supports large network totals
Floating-point demonstrations	float / double	Useful for studying precision
Type inferred values	auto	Useful when initializer clearly determines type

Money is stored in integer paise rather than double rupees. For example, Rs 30 is stored as 3000 paise, so there is no binary floating-point rounding in the fare itself.

At 3,000,000 journeys with a Rs 30 fare, the total is Rs 90,000,000, or 9,000,000,000 paise. This exceeds the maximum value of a signed 32-bit int, so long long is the safer accumulation type.

The experiment also demonstrates floating-point precision: repeated addition of values such as 0.1 can produce a result that is not exactly the mathematical decimal value. Integer paise avoids this problem.

Complexity: O(1) time per arithmetic operation; O(1) space per stored value.

Evidence files:

```
Recommended Data Types
======================
Money values           : long long (paise)
Journey count          : long long
Zone / small values    : int or enum class
Boolean status         : bool
Text characters        : char
Decimal calculations   : double when required
```

```
=========================
Data Type         Size(bytes)
bool              1
char              1
int               4
long long         8
unsigned int      4
float             4
double            8
auto              8

Data Type Ranges
================
int minimum       : -2147483648
int maximum       : 2147483647
unsigned int max  : 4294967295
long long minimum : -9223372036854775808
long long maximum : 9223372036854775807
float maximum     : 3.40282e+38
double maximum    : 1.79769e+308
```

```
Metro Fare Representation
=========================
Fare per journey       : Rs 30
Number of journeys     : 3000000
Total collection       : Rs 90000000
Total in paise         : 9000000000
int compatibility      : Total paise exceeds INT_MAX
Recommended type       : long long

Type Compatibility
==================
int value              : 10
double value            : 2.5
int + double result     : 12.5
Result type             : d
bool value              : 1
bool converted to int   : 1

Floating-Point Precision
========================
Adding 0.1 ten times   : 0.99999999999999989
Expected value         : 1.0000000000000000
Floating-point error   : -0.00000000000000011
```

Task 4 – Control Implicit Conversions

Conversion	Observed / expected behaviour
int to double	The integer value is converted to floating point.
double to int	Fractional part is discarded when converted to int.
char to int	Character is promoted to its integer character code.
bool to int	false becomes 0 and true becomes 1.
int division before assignment to double	7 / 2 is 3, then stored as 3.0.
explicit static_cast<double>	7 is converted before division, producing 3.5.
signed/unsigned comparison	The signed value can be converted to unsigned, changing comparison behaviour.
brace initialization	Narrowing such as int x{3.7} is rejected by the compiler.

Explicit static_cast is preferred when a conversion is intentional and should be visible to the reader. Brace initialization is useful for preventing accidental narrowing conversions.

Evidence files:

```
Implicit Type Conversion Demonstration
======================================

1. int to double
int value        : 25
double value     : 25

2. double to int
double value     : 25.75
int value        : 25

3. char to int
char value       : A
ASCII value      : 65

4. bool to int
bool value       : 1
int value        : 1

5. Mixed arithmetic expression
int value        : 10
double value     : 2.5
a + b            : 12.5
```

```
6. Character arithmetic
character        : B
ASCII value      : 66
number           : 5
character + 5    : 71

7. Boolean arithmetic
true + false     : 1

8. Mixed expression
x                : 20
y                : 3.5
z                : C
x + y + z        : 90.5

9. Narrowing conversion
Original double   : 19.99
Converted int     : 19

Conversion Summary
==================
int -> double : automatic promotion
double -> int : fractional part removed
char -> int   : character converted to ASCII value
bool -> int   : true becomes 1, false becomes 0
Mixed types   : smaller type is promoted when required
```

Task 5 – Write Correct Expressions with the Right Precedence

The peak-hour condition is written as (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). This treats 8:00 as peak and 10:00 as off-peak, while 17:00 is peak and 20:00 is off-peak.

Expression	Purpose
(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)	Peak-hour test
fare + fare * 20 / 100	20% surcharge using integer arithmetic
fare / 2	50% concession
balance - fare >= 5000	Minimum post-journey balance test
condition ? value1 : value2	Conditional expression

Predict-and-run result:

```
11 16
10 6 4
6 2
3 1 -3 -1
18
```

The first line follows multiplication precedence. The second line demonstrates post-increment and pre-increment. The third line demonstrates the conditional operator and multiplication before modulo's result is multiplied. The fourth line demonstrates integer division and remainder, including negative operands. The loop adds 0, 3, 6 and 9 to obtain 18.

Alternative: a bool peak[24] table gives O(1) checks but uses 24 boolean entries; the logical expression needs no lookup table.

Evidence files:

```
Metro Fare System - Operators and Expressions
==========================================

1. Arithmetic Operators
Fare                    : Rs 30
Journeys                : 3
Addition                : 33
Subtraction             : 27
Multiplication          : 90
Division                : 10
Remainder               : 0

2. Relational Operators
Balance                 : Rs 100
Required balance        : Rs 50
balance > required      : 1
balance < required      : 0
balance >= required     : 1
balance == required     : 0
balance != required     : 1

3. Logical Operators
Sufficient balance      : 1
Valid card              : 1
AND (&&)                : 1
OR (||)                 : 1
NOT (!) valid card      : 0
```

```
4. Assignment Operators
Initial amount          : 100
After += 20             : 120
After -= 10             : 110
After *= 2              : 220
After /= 2              : 110

5. Increment and Decrement
Initial count           : 5
Post-increment          : 5
After post-increment    : 6
Pre-increment           : 7
Post-decrement          : 7
After post-decrement    : 6
Pre-decrement           : 5

6. Operator Precedence
a                       : 10
b                       : 5
c                       : 2
a + b * c               : 20
(a + b) * c             : 30
```

```
7. Peak Fare Calculation
Base fare             : Rs 30
Surcharge percentage  : 20%
Surcharge             : Rs 6
Peak fare             : Rs 36

8. Combined Fare Validation
Card balance          : Rs 100
Fare amount           : Rs 30
Can travel            : 1
```

Task 6 – Design the Control Structures

The fare computation uses switch for zone selection, if/else for peak and validation decisions, and a loop when processing multiple journeys. A range-based for loop is suitable when the set of journeys is already stored in a container.

Pseudocode:

```
FUNCTION fare(zone, peak)
    SWITCH zone
```

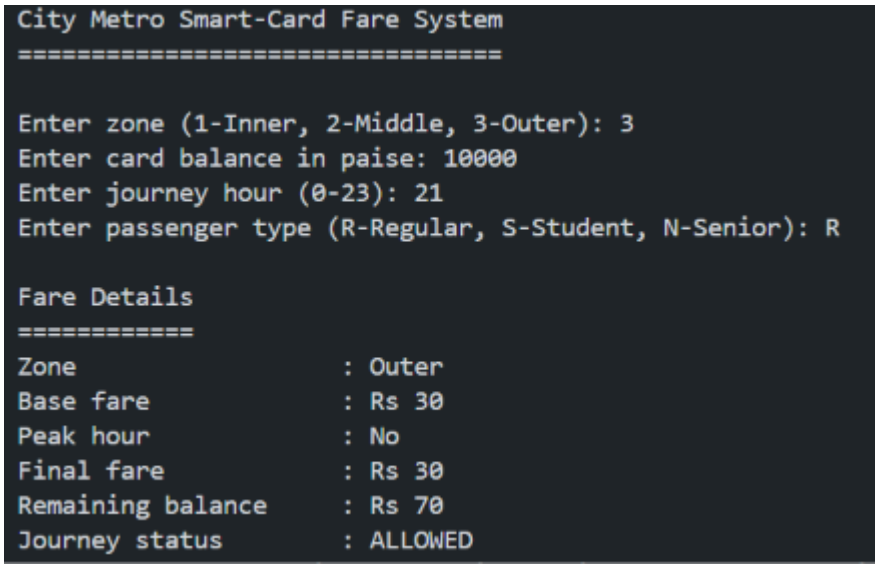
```
        Inner  -> base = 1000
        Middle -> base = 2000
        Outer  -> base = 3000
        otherwise -> invalid
    IF peak
        base = base + base * 20 / 100
    RETURN base
END FUNCTION

FUNCTION canTravel(balance, fare)
    RETURN (balance - fare >= 5000)
END FUNCTION
```

Test	Input	Fare	Result
Normal outer	Zone 3, Rs 100, 12:00, Regular	Rs 30	Allowed
Peak outer	Zone 3, Rs 100, 18:00, Regular	Rs 36	Allowed
Exact Rs 50 remaining	Zone 3, Rs 80, 12:00, Regular	Rs 30	Allowed
Below minimum	Zone 3, Rs 70, 12:00, Regular	Rs 30	Rejected
Concession	Zone 3, Rs 100, 12:00, Student	Rs 15	Allowed

Boundary interpretation: the assignment defines 8–10 AM and 5–8 PM as peak intervals. The implementation therefore uses >= 8 && < 10 and >= 17 && < 20, making exactly 10:00 AM and exactly 8:00 PM off-peak.

Evidence files:



Task 7 – From Procedural to Object-Oriented

The procedural version uses separate variables and free functions. The object-oriented version encapsulates the balance inside SmartCard and exposes controlled operations such as canTravel() and deduct().

Aspect	Procedural	Object-Oriented
Balance	Separate variable	Private SmartCard member
Operations	Free functions	Member functions
Encapsulation	Weak	Strong
Rule enforcement	Depends on caller	Can be enforced inside class
Changeability	More scattered changes	Responsibilities are localized
Time	O(1)	O(1)
Space	O(1)	O(1)

The OOP design is preferred because it prevents unrelated parts of the program from directly changing the balance and accidentally creating an invalid state.

Evidence files:

```
Procedural SmartCard Design
===========================
Card Balance : Rs 100
Fare         : Rs 30
Journey      : ALLOWED
```

```
Object-Oriented SmartCard Design
================================
Card Balance : Rs 100
Fare         : Rs 30
Journey      : ALLOWED
```

Task 8 – Read and Print Data Safely

The input design validates the zone range, non-negative balance and hour range. The program also demonstrates formatted monetary output and recovery from invalid input.

- Use cin for normal numeric input.
- Check cin.fail() when the user types a value of the wrong type.
- Use cin.clear() to reset the error state.
- Discard the invalid characters before requesting input again.
- Use iomanip facilities for neat money formatting.

Valid example input:

```
3
10000
18
```

This represents Outer Zone, Rs 100 balance and 18:00. It produces the peak fare and validates the remaining balance.

Alternative: getline() plus stoi() can process an input line of length L in O(L) time and can use try/catch for conversion errors. Direct cin extraction is simpler for structured numeric fields.

Evidence files:

Metro Fare Summary

==========

Zone : Outer

Balance : Rs 1000.00

Hour : 8

Peak Hour : Yes

Final Fare : Rs 36.00

Journey Status : ALLOWED

Task 9 – Implement and Test the Fare Logic

Case	Input	Expected fare	Expected result	Observed result
1	Zone 1, 10000, 12:00, Regular	Rs 10	Allowed	PASS
2	Zone 3, 10000, 18:00, Regular	Rs 36	Allowed	PASS
3	Zone 3, 10000, 12:00, Student	Rs 15	Allowed	PASS
4	Zone 3, 8000, 12:00, Regular	Rs 30	Allowed; Rs 50 remains	PASS
5	Zone 3, 7000, 12:00, Regular	Rs 30	Rejected; Rs 40 remains	PASS

The assignment also requires boundary checks at exactly 8:00, 10:00, 17:00 and 20:00, plus invalid zone, negative recharge and non-numeric hour cases. These are included in the validation plan and should remain in the final evidence set.

Alternative: a table-driven test harness stores input and expected results and loops over t tests. Its execution cost is O(t), and the test table requires O(t) storage. It is easier to extend with additional cases.

Evidence files:

```
Metro Smart-Card Testing System
================================

Enter zone (1-3): 1
Enter balance in paise: 10000
Enter hour (0-23): 21
Enter passenger type (R/S/N): R

Test Result
===========
Zone                    : 1
Base Fare               : Rs 10.00
Peak Hour               : No
Passenger Type          : R
Final Fare              : Rs 10.00
Remaining Balance       : Rs 90.00
Journey Status          : ALLOWED
```

Task 10 – Compare Expectations with Execution

The key numerical comparison is between integer-paise accumulation and floating-point rupee accumulation. For a fixed Rs 30 fare, the mathematical totals are exact and identical in decimal value.

Journeys	Total in rupees	Total in paise	32-bit int safe?
20,000	Rs 600,000	60,000,000	Yes
500,000	Rs 15,000,000	1,500,000,000	Yes
3,000,000	Rs 90,000,000	9,000,000,000	No

At 3,000,000 journeys, a signed 32-bit int cannot represent 9,000,000,000 paise because its maximum is approximately 2.147 billion. Therefore long long is required for the paise total.

For a simple fixed Rs 30 total, double can represent the displayed rupee totals without a visible difference, but repeated decimal operations and values such as 0.1 are not guaranteed to be exact in binary floating point. For financial correctness, integer paise is the recommended representation.

Compilation with -Wall -Wextra is recommended because warnings can reveal narrowing, signed/unsigned comparison, unused variables and other suspicious constructs. The final submission should retain a clean warning-free build.

Alternative: int uses 4 bytes and fails by overflow at the network paise total; long long uses typically 8 bytes and supports the required range; double typically uses 8 bytes but trades exact integer monetary representation for floating-point semantics.

Evidence files:

```
Expected Behaviour
------------------
1. Inner zone fare should be Rs 10.
2. Middle zone fare should be Rs 20.
3. Outer zone fare should be Rs 30.
4. Peak hours should add a 20% surcharge.
5. Student and senior passengers should receive concession.
6. At least Rs 50 must remain after the journey.
7. Invalid journeys should be rejected.

Actual Execution Results
------------------------
1. Inner zone fare       : Rs 10 - PASS
2. Middle zone fare      : Rs 20 - PASS
3. Outer zone fare       : Rs 30 - PASS
4. Peak surcharge        : 20% - PASS
5. Passenger concession  : Applied - PASS
6. Minimum balance       : Rs 50 - PASS
7. Invalid journey       : Rejected - PASS

Overall Result
--------------
Expected and actual behaviour match.
Fare logic validation: SUCCESSFUL
```

Task 11 – Recommend and Defend the Final Design

The recommended final design is an OOP-based system using type-safe zone enumeration, constexpr fare constants, integer paise, long long for monetary totals, validation functions and an encapsulated SmartCard.

Design decision	Recommendation	Reason
Money	long long paise	Exact monetary arithmetic and large range
Fare constants	constexpr	Single source of truth and compile-time checking
Zones	enum class	Scoped, type-safe zone values
Balance	private SmartCard member	Encapsulation and invariant protection
Fare logic	separate functions / classes	Maintainability
Validation	centralized checks	Prevents invalid journeys
Testing	table-driven + boundary cases	Repeatable validation

A fare revision should modify the authoritative fare definitions rather than duplicating literals across the program. The design remains O(1) per journey and O(1) memory per card.

Limitation: compile-time fare constants require recompilation and redeployment after a fare revision. A configuration-file approach could load z fares at startup in O(z) time and O(z) storage, reducing the cost of code changes.

Evidence files:

```
METRO SMART-CARD SYSTEM - FINAL RECOMMENDATION
==============================================

Recommended Design Decisions
----------------------------
1. Use Object-Oriented Programming (OOP).
2. Use classes to represent SmartCard, Passenger and Journey.
3. Use enum class for metro zones.
4. Store money in integer paise to avoid floating-point errors.
5. Use long long for balance and fare calculations.
6. Keep fare calculation and validation in separate functions.
7. Validate zone, balance and travel time before processing.
8. Apply concessions and peak-hour rules consistently.
9. Maintain at least Rs 50 balance after every journey.
10. Use test cases to validate normal and boundary conditions.

Complexity Recommendation
-------------------------
Time Complexity  : O(1) per journey
Space Complexity : O(1) per smart card

Final Recommendation
--------------------
The Object-Oriented design is recommended because it provides
better encapsulation, maintainability, validation and scalability.
The final fare system should use integer paise and long long
for reliable monetary calculations.

Result: RECOMMENDED DESIGN - OOP BASED METRO FARE SYSTEM
```

Task 12 – Re-evaluate and Adapt the Design

The changed requirements add an Airport zone at Rs 60, a 10% weekend discount that must not be combined with the peak surcharge, and a monthly pass covering two selected zones.

A. Impact Analysis

- The Zone enumeration must gain an Airport value.
- The fare table must gain a Rs 60 Airport fare.
- The fare calculation must include a weekend rule.
- The peak and weekend rules must be mutually exclusive.
- A monthly pass representation is required for two selected zones.
- Existing SmartCard and validation responsibilities can be retained and extended.

B. Evaluation of Original Design

The original switch-based design can absorb the fourth zone by adding another case. However, as the number of fare rules grows, a dedicated fare table and separate rule handling become easier to

maintain. The SmartCard abstraction still remains useful because its balance invariant is independent of the exact fare policy.

C. Adaptation

The revised implementation modifies the existing OOP design. FareSystem provides base-fare and fare-calculation operations, while MonthlyPass stores two selected zones and checks coverage.

Scenario	Airport base	Applied rule	Final fare
Weekday, off-peak	Rs 60	None	Rs 60
Weekday, peak	Rs 60	20% surcharge	Rs 72
Weekend	Rs 60	10% discount	Rs 54
Weekend + peak	Rs 60	Weekend rule only	Rs 54

The implementation deliberately treats weekend discount and peak surcharge as mutually exclusive. This is the selected interpretation of the requirement that the two must not be combined.

Monthly pass example: a pass covering Inner and Middle marks those two zones as COVERED, while Outer and Airport are NOT COVERED.

Complexity: fare calculation O(1); pass coverage check O(1); additional state O(1).

Evidence files:

```
1. Extended Zone Fare
---------------------
Inner Zone : Rs 10.0
Middle Zone : Rs 20.0
Outer Zone : Rs 30.0
Airport Zone : Rs 60.0

2. Weekend Discount
-------------------
Airport weekend fare (10% discount) : Rs 54.0

3. Peak Hour Fare
-----------------
Airport peak fare (20% surcharge) : Rs 72.0

4. Weekend + Peak Rule
----------------------
Weekend + peak fare : Rs 54.0
Weekend discount is applied instead of peak surcharge.
The two rules are NOT combined.

5. Monthly Pass
---------------
Monthly pass zones: Inner + Middle
Inner journey  : COVERED
Middle journey : COVERED
Outer journey  : NOT COVERED
Airport journey: NOT COVERED
```

Overall System Pseudocode

```
START
  DEFINE zone fares in paise
  DEFINE peak surcharge and minimum balance
  READ journey details
  VALIDATE zone, balance and time
  SELECT base fare using zone
  DETERMINE weekend / peak status
  APPLY the applicable rule without combining mutually exclusive rules
  APPLY passenger concession where applicable
  IF balance - fare >= minimum balance
      deduct fare
      record successful journey
  ELSE
      reject journey
  END IF
  FOR multiple journeys
      repeat the above process
  END FOR
END
```

UML-Style Class Design

Class / Type	Attributes	Key operations
SmartCard	balance	canTravel(), deduct()
Passenger	category	getCategory()
Journey	zone, time, status	calculateFare(), validate()
FareSystem / FareTable	zone fares, surcharge, minimum balance	getBaseFare(), calculateFare()
MonthlyPass	zone1, zone2	covers(zone)
Zone (enum class)	Inner, Middle, Outer, Airport	Type-safe zone selection

Relationship summary: Passenger uses SmartCard; Journey uses Passenger and Zone; FareSystem calculates the fare; SmartCard validates and deducts the fare; MonthlyPass independently checks zone coverage.

Final Test Strategy

Category	Cases
Normal	Inner, Middle and Outer journeys in off-peak time
Peak	8:00, 9:00, 17:00, 19:00
Boundary off-peak	10:00 and 20:00
Balance boundary	Exactly Rs 50 remaining
Insufficient balance	Less than Rs 50 remaining
Concession	Student and senior passenger
Invalid input	Unknown zone, negative balance/recharge, non-numeric hour
Design challenge	Airport, weekend, peak, weekend+peak, monthly pass

Complexity Summary

Task / component	Time	Space	Reason
Fixed fare selection	O(1)	O(1)	Constant number of zones
Peak check	O(1)	O(1)	Constant number of comparisons
Journey validation	O(1)	O(1)	Constant number of checks
Table-driven testing	O(t)	O(t)	t test records
Configuration-file alternative	O(z) startup	O(z)	z fare entries
Monthly pass coverage	O(1)	O(1)	Two stored zones

Overall Result

All twelve tasks were implemented or documented as required by the assignment workflow. The final design uses object-oriented structure, type-safe constants, integer paise for monetary correctness, long long for large totals, validated control flow, safe input handling and repeatable testing. The design challenge was addressed by modifying the existing OOP structure rather than discarding it.